

BoxVoyage

Documentation Technique

Projet de fin de L3 Informatique — Box Certificative Finale

Chahine Bensafia

Ilan Sarahoui

Victorian Da Cunha

2025/2026

1. Contexte du projet

BoxVoyage est une application console en Python permettant à des utilisateurs de gérer des lieux d'intérêt géographiques et de construire des circuits de voyage optimisés. Le projet a été développé dans le cadre de la Box Certificative Finale de L3 Informatique.

1.1 Objectifs fonctionnels

- Authentification sécurisée (inscription / connexion)
- Gestion d'une liste de lieux personnels avec géocodage automatique
- Création et gestion de circuits (tours) multi-étapes
- Optimisation de l'ordre de visite des villes (algorithme TSP)
- Gestion d'hôtels et d'excursions journalières depuis un hôtel
- Partage de circuits publics entre utilisateurs

1.2 Interprétations du sujet

Le sujet mentionne « optimisation d'un circuit » sans préciser l'algorithme. Nous avons choisi une approche en deux phases : Nearest Neighbor pour construire rapidement une solution initiale, puis 2-opt pour l'améliorer localement. Cette combinaison offre un bon équilibre entre qualité de la solution et temps de calcul pour des instances de taille courante (< 50 villes).

Le concept d'hôtel n'étant pas explicitement défini, nous l'interprétons comme une ville base depuis laquelle le voyageur effectue des excursions journalières vers des villes proches (rayon configurable, défaut 60 km aller simple). Les villes d'excursion restent dans la liste des places afin de ne pas être perdues si l'hôtel est retiré ultérieurement.

2. Architecture du système

2.1 Vue d'ensemble — architecture en 3 couches

Le projet suit une séparation stricte en trois couches :

Couche	Package	Rôle
Présentation	src/UI/	Interface console, menus, saisies utilisateur
Métier (core)	src/core/	Logique applicative, algorithmes, entités
Données	src/data/	Lecture/écriture JSON, abstraction du stockage

2.2 Structure des fichiers

```
BoxVoyage/
├── main.py                # Point d'entrée
├── src/
│   ├── UI/
│   │   └── console.py    # Interface console (menus, saisies)
│   ├── core/
│   │   ├── algorithm.py  # GeoPoint, DistanceCalculator, TourOptimischer
│   │   ├── auth.py       # Users, AuthManager
│   │   ├── places.py     # Place, GeocodingService, PlaceManager
│   │   └── tours.py      # Tour, TourManager
│   └── data/
│       └── storage.py    # JsonStorage
└── dataBase/
    ├── users.json
    ├── places.json
    └── tours.json
```

2.3 Diagramme de dépendances

Les dépendances entre modules suivent la règle : les couches supérieures dépendent des couches inférieures, jamais l'inverse.

```
console.py
├── auth.py      (AuthManager)
├── places.py    (PlaceManager, GeocodingService)
├── tours.py     (TourManager)
└── algorithm.py (TourOptimischer)

places.py → algorithm.py (GeoPoint)
tours.py  → places.py    (Place)
auth.py, places.py, tours.py → storage.py (JsonStorage)
```

2.4 Format de stockage JSON

Chaque fichier JSON est un dictionnaire indexé par nom d'utilisateur.

```
# tours.json
{
```

```
"alice": [  
  {  
    "id": "a1b2c3d4",  
    "name": "Tour Europe",  
    "owner": "alice",  
    "is_public": false,  
    "places": [ {"name": "Paris", "lat": 48.85, "lon": 2.35, "owner": "alice"} ],  
    "hotels": ["Paris"]  
  }  
]  
}
```

3. Choix technologiques

Technologie	Usage	Justification
Python 3.10+	Langage principal	Syntaxe claire, bibliothèques riches, imposé par le sujet
geopy / Nominatim	Géocodage	API gratuite OpenStreetMap, aucune clé requise
json (stdlib)	Persistance	Format lisible, suffisant pour un usage mono-utilisateur
hashlib (stdlib)	Sécurité mots de passe	SHA-256 disponible nativement, pas de dépendance externe
uuid (stdlib)	Identifiants tours	Génère des IDs uniques sans base de données

Le choix d'un stockage JSON plutôt qu'une base de données relationnelle est délibéré : le projet est mono-poste, les volumes de données sont faibles, et JSON offre une inspection visuelle facile (utile pour le développement et le débogage).

4. Description des algorithmes

4.1 Distance orthodromique (grande-cercle)

Fichier : `src/core/algorithm.py` — classe `DistanceCalculator.distance()`

Pour calculer la distance réelle entre deux points GPS, nous utilisons la formule de la grande-cercle (orthodromie) :

```
D = R * arccos( sin(lat1) * sin(lat2) + cos(lat1) * cos(lat2) * cos(lon2 - lon1) )
```

Avec $R = 6\,378.197$ km (rayon terrestre), et les angles en radians.

Toutes les coordonnées sont stockées en degrés décimaux (DD) et converties en radians à la volée via les propriétés `lat_rad` / `lon_rad` de `GeoPoint`. Un clamping dans $[-1, 1]$ est appliqué avant `arccos` pour éviter des erreurs numériques sur des points quasi-identiques (`cos` légèrement > 1.0).

4.2 Nearest Neighbor (heuristique)

Fichier : `TourOptimiser.nearest_neighbor()`

Objectif : construire rapidement un ordre de visite « raisonnable » pour le problème du voyageur de commerce (TSP).

Fonctionnement :

- On part d'une ville de départ (par défaut la première de la liste).
- À chaque étape, on va vers la ville non encore visitée la plus proche.
- On répète jusqu'à avoir visité toutes les villes, puis on retourne à la ville de départ.

Complexité : $O(n^2)$. Le résultat n'est pas optimal mais sert de bonne solution initiale pour l'étape 2-opt. La liste retournée est fermée : `[A, B, C, A]`.

4.3 Amélioration 2-opt

Fichier : `TourOptimiser.two_opt()`

Objectif : améliorer itérativement le tour produit par Nearest Neighbor en supprimant les croisements de segments.

Principe :

Pour chaque paire d'indices (i, k) , on inverse le segment $[i..k]$ du tour et on vérifie si la distance totale diminue. Si oui, on conserve le nouveau tour et on recommence depuis le début. On s'arrête quand aucun échange n'améliore le résultat.

Exemple concret :

```
Tour initial : [Paris, Lyon, Marseille, Bordeaux, Paris]
```

```
Segments qui se croisent :
```

```
Paris → Marseille (long) et Lyon → Bordeaux (long)
```

```
Après inversion du segment [Lyon, Marseille] :  
[Paris, Marseille, Lyon, Bordeaux, Paris] ← distance totale réduite
```

Complexité : $O(n^2)$ par itération, $O(n^3)$ dans le pire cas. Pour $n < 50$ villes, l'exécution est quasi-instantanée.

Important : 2-opt ne garantit pas l'optimum global — il converge vers un optimum local (aucune inversion de segment ne peut plus améliorer le tour). Pour un optimum global, il faudrait des algorithmes exacts (branch & bound, programmation dynamique) dont la complexité est exponentielle.

4.4 Pipeline complet d'optimisation

Fichier : `TourOptimiser.optimize_places_with_distances()`

Cette fonction orchestre les étapes en une seule appel :

- Conversion des dicts de places en objets `GeoPoint`
- Nearest Neighbor → tour fermé `[A...Z, A]`
- Suppression du doublon final `[:-1]` avant 2-opt (2-opt travaille sur un tour ouvert)
- 2-opt → tour amélioré
- Reconstruction des dicts de places depuis les `GeoPoints` optimisés
- Calcul des distances par segment et de la distance totale aller-retour

La fonction retourne un dict avec clés : `optimized_places`, `segments`, `total_distance`.

4.5 Gestion des hôtels et excursions

Fichier : `TourOptimiser.split_by_hotels()`

Lorsque des villes sont désignées comme hôtels, l'optimisation se déroule en deux temps :

Étape 1 — Partitionnement (style Voronoï) :

- Pour chaque ville non-hôtel, on calcule sa distance à chaque hôtel.
- Si la distance au plus proche hôtel est $\leq$ rayon (défaut 60 km), la ville devient une excursion de cet hôtel.
- Sinon, elle reste dans le tour principal.
- Les hôtels eux-mêmes font toujours partie du tour principal.

Étape 2 — Optimisation du tour principal uniquement :

- On appelle `optimize_places_with_distances` sur la liste `main_tour` uniquement.
- Les villes d'excursion ne participent pas à la boucle principale.

Affichage :

- Chaque hôtel est affiché dans l'ordre du tour principal.
- Ses excursions sont affichées en retrait (`↳`) avec la distance aller-retour.

Les villes d'excursion sont conservées dans places du tour (après les villes principales). Si l'hôtel est retiré et que l'on recalcule, elles réintègrent automatiquement le tour principal.

5. Sécurité

5.1 Hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. À l'inscription, le mot de passe est immédiatement haché avec SHA-256 (hashlib) et seul le hash hexadécimal est persisté dans users.json.

```
import hashlib
password_hash = hashlib.sha256(password.encode()).hexdigest()
# Résultat : chaîne hex de 64 caractères
```

À la connexion, le même hachage est appliqué au mot de passe saisi et comparé au hash stocké. Le texte en clair ne transite jamais vers le disque.

5.2 Bypass de `__init__` à la désérialisation

Lors du chargement depuis JSON, les classes Users et Tour utilisent `cls.__new__(cls)` pour contourner `__init__`. Cela évite que le constructeur re-hache un hash déjà haché (Users) ou génère un nouvel UUID (Tour). Les attributs sont ensuite assignés manuellement depuis le dictionnaire JSON.

6. Gestion du code source

6.1 Branches Git

Le développement a été organisé en deux branches principales :

- main : branche stable, fonctionnalités de base.
- Chahine : branche de développement, fonctionnalités avancées (hôtels, excursions, ville de départ, commentaires de code).

Un conflit de fusion (merge conflict) sur console.py a été résolu manuellement : la branche collègue avait ajouté le choix de la ville de départ (menu entrée 5), notre branche avait ajouté la gestion des hôtels. Le menu final intègre les deux fonctionnalités : 5=ville de départ, 6=visibilité, 7=hôtels, 8=retour.

6.2 Convention d'imports

Tous les imports utilisent le préfixe src. (from src.core.algorithm import ...) car le projet est exécuté depuis la racine avec python main.py. Cela garantit que les chemins relatifs sont cohérents quel que soit l'OS.

Note : sous Linux, le système de fichiers est sensible à la casse. Les répertoires src/core/ et src/Core/ sont distincts. Le projet utilise src/core/ (minuscule) de façon cohérente.

7. Instructions d'installation et de mise en route

7.1 Prérequis

- Python 3.10 ou supérieur
- pip (inclus avec Python)
- Accès Internet (pour le géocodage via Nominatim)

7.2 Installation

```
# 1. Cloner le dépôt
git clone <url_du_dépôt>
cd BoxVoyage

# 2. Installer les dépendances
pip install geopy

# 3. Lancer l'application
python main.py
```

7.3 Arborescence de données

Au premier lancement, le dossier dataBase/ est créé automatiquement avec des fichiers JSON vides. Aucune configuration manuelle n'est requise.

7.4 Utilisation rapide

Déroulement typique d'une session :

1. S'inscrire ou se connecter.
2. Ajouter des lieux (menu principal → option 1).
3. Créer un tour depuis sa liste de lieux (option 2).
4. Dans le tour : ajouter des villes, désigner des hôtels, optimiser.
5. Consulter le résumé avec distances par segment.

8. Limites et pistes d'amélioration

8.1 Limites connues

- Stockage JSON non concurrent : si deux processus modifient le fichier simultanément, des données peuvent être perdues.
- SHA-256 sans sel (salt) : deux utilisateurs avec le même mot de passe auront le même hash. En production, bcrypt ou argon2 avec sel seraient préférables.
- Géocodage réseau : si Nominatim est indisponible, l'ajout de lieux échoue silencieusement. Un mécanisme de retry ou de cache améliorerait la robustesse.
- 2-opt : optimum local seulement. Pour de très grands tours, Lin-Kernighan ou des algorithmes génétiques donneraient de meilleurs résultats.
- Interface console uniquement : pas d'interface graphique ni d'API REST.

8.2 Évolutions possibles

- Ajouter une interface web (Flask/FastAPI) en conservant la couche core intacte.
- Remplacer JSON par SQLite pour une meilleure concurrence et des requêtes plus riches.
- Intégrer une carte interactive (Folium) pour visualiser les tours optimisés.
- Ajouter des tests unitaires (pytest) pour les algorithmes et le stockage.